```
$count = count($rogueAgents);
for ($i = 0; $i < $count; $i++) {
    print $rogueAgents[$i] . PHP_EOL;
}

foreach ($rogueAgents as $agent) {
    print $agent . PHP_EOL;
}

Output:
Aaron Keener
Theo Parnell
Alani Kelso
```

The output of both `for` and `foreach` loops is identical and given once above. Don't forget the `break` keyword (also shown in Chapter 5, "Array," in addition to the similar `continue` keyword) allows early loop termination:

```
foreach ($rogueAgents as $agent) {
    if ($agent === 'Alani Kelso') {
        break;
    }
    print $agent . PHP_EOL;
}

Output:
Aaron Keener
Theo Parnell
```

I guess Kelso might not be as rogue as the other agents… Anyway, for more detail you can always refer back to Chapter 5, "Array." Moving on from iterating structures, we have looping structures that are predicated on logical expressions.

`while`, `do-while`

The simplest looping construct is `while`. It takes an expression, which, if it evaluates to `true`, moves execution into a block of statements: